

Design and Implementation of a 6-DOF Stewart Platform on an Embedded Microcontroller

Wm. Sam Schneidereith
University of Maryland, College Park

This paper presents the design, kinematic analysis, and embedded software implementation of a six-degree-of-freedom (6-DOF) Stewart Platform actuated by six RC hobby servos. The system runs on an ESP32 microcontroller and communicates with a PCA9685 16-channel PWM driver over I²C to generate servo control signals. A numerical inverse kinematics (IK) solver computes the required horn angles from a desired platform pose represented as a translation and ZYX Euler rotation. Smooth trajectories are achieved through linear pose interpolation at 50 Hz. Two continuous motion sequences—flat circular precession and angled gyroscopic precession—are implemented and demonstrated. The modular, layered firmware architecture enables straightforward extension to new geometries and motion profiles.

I. Introduction

A Stewart Platform is a class of parallel manipulator capable of motion in all six degrees of freedom: translation along the x , y , and z axes, and rotation about those same axes (roll, pitch, and yaw) [1]. Unlike serial manipulators, which chain joints end-to-end, a Stewart Platform connects its moving plate (the *platform*) to a fixed base through six independent actuated legs arranged in parallel. This topology provides inherently high structural stiffness, good load distribution across all actuators, and precise positional control.

Parallel manipulators of this class see widespread use in flight simulators, camera stabilization rigs, precision machining fixtures, and haptic interfaces. Implementing a small-scale version using commodity hobby-grade components offers a practical and low-cost testbed for studying robotic kinematics, real-time embedded control, and motion planning algorithms without access to industrial hardware.

This paper is organized as follows. Section II describes the mechanical design and hardware configuration. Section III derives the forward and inverse kinematics. Section IV presents the software architecture. Section V discusses system performance and directions for future work.

II. Mechanical Design and Hardware

A. Platform Geometry

The platform consists of two approximately hexagonal plates—a fixed base and a moving platform—connected by six rigid rods. Each rod attaches at the base end to a servo horn tip and at the platform end to a ball-and-socket joint. The geometry is parameterized by the values listed in Table 1.

Table 1 Stewart Platform Geometric Parameters

Parameter	Symbol	Value
Rod length (rod + rod end)	L	190 mm
Horn length	r	30 mm
Base joint radius	R_b	88.9 mm
Base joint height below datum	h_b	−19 mm
Platform joint radius	R_p	74.1 mm
Platform joint height below platform	h_p	−2.5 mm

The six servo pivot points on the base are arranged in three pairs, each pair straddling a primary symmetry axis.

Their azimuth angles are:

$$\theta_{\text{base}} = \left\{ \frac{15\pi}{12}, \frac{17\pi}{12}, \frac{23\pi}{12}, \frac{\pi}{12}, \frac{7\pi}{12}, \frac{9\pi}{12} \right\} \quad [\text{rad}] \quad (1)$$

The platform joints are similarly arranged in offset pairs so that each leg connects a base servo to its nearest platform ball joint, minimizing lateral bending forces on the servo horns.

B. Servo and Driver Hardware

Six RC servo motors serve as actuators. Each servo is connected to one channel of a PCA9685 16-channel PWM driver, which communicates with the ESP32 over I²C at 400 kHz. The PCA9685 is configured to output a 50 Hz PWM signal, consistent with the standard RC servo control protocol [2].

Each servo has individually calibrated minimum and maximum pulse widths to account for unit-to-unit variation, and an inversion flag to handle the alternating physical orientation of servos around the base ring. The per-servo configuration is listed in Table 2.

Table 2 Per-Servo PWM Calibration and Configuration

Servo	Channel	Min Pulse (μs)	Max Pulse (μs)	Inverted
1	0	450	2400	Yes
2	1	400	2400	No
3	2	430	2400	Yes
4	3	420	2350	No
5	4	550	2490	Yes
6	5	400	2300	No

Every other servo faces the opposite direction around the base ring. The inversion flag causes the software to negate the commanded angle for those channels, so a positive angle always lifts the corresponding horn tip in the same upward sense regardless of physical orientation.

III. Kinematics

A. Coordinate System and Pose Representation

The platform pose is represented as a 6-vector:

$$\mathbf{q} = [x \quad y \quad z \quad \phi \quad \theta \quad \psi]^T \quad (2)$$

where (x, y, z) is the translation of the platform centroid relative to the base centroid in meters, and (ϕ, θ, ψ) are roll, pitch, and yaw in radians. The rotation matrix is constructed using the ZYX Euler convention:

$$\mathbf{R}(\phi, \theta, \psi) = \mathbf{R}_z(\psi) \mathbf{R}_y(\theta) \mathbf{R}_x(\phi) \quad (3)$$

B. Platform Joint Positions

Given a pose $\mathbf{q}$, the world-frame position of the i -th platform joint is:

$$\mathbf{P}_i = \mathbf{R} \mathbf{p}_i + \mathbf{T} \quad (4)$$

where $\mathbf{p}_i$ is the joint position in the platform body frame and $\mathbf{T} = [x, y, z]^T$.

C. Inverse Kinematics

Given a desired pose, the inverse kinematics (IK) problem is to find the six servo horn angles $\alpha_i \in [-45, +45]$ such that each rod exactly spans the distance between the corresponding horn tip and platform joint.

1. Horn Tip Kinematics

Each servo horn rotates about the servo axle. The tangential unit vector perpendicular to the radial direction in the horizontal plane is:

$$\hat{\mathbf{r}}_i = \frac{1}{\|\mathbf{b}_i^{xy}\|} \begin{bmatrix} b_{ix} \\ b_{iy} \\ 0 \end{bmatrix}, \quad \hat{\mathbf{t}}_i = \begin{bmatrix} -\hat{r}_{iy} \\ \hat{r}_{ix} \\ 0 \end{bmatrix} \quad (5)$$

For inverted servos $\hat{\mathbf{t}}_i$ is negated. The horn tip position as a function of angle α is then:

$$\mathbf{H}_i(\alpha) = \mathbf{b}_i + \hat{\mathbf{t}}_i (r \cos \alpha) + \hat{\mathbf{z}} (r \sin \alpha) \quad (6)$$

where $\mathbf{b}_i$ is the servo pivot, r is the horn length, and $\hat{\mathbf{z}} = [0, 0, 1]^T$.

2. Numerical Search

The IK solver minimizes the residual between the actual rod length and the target rod length L :

$$e_i(\alpha) = \left| \|\mathbf{P}_i - \mathbf{H}_i(\alpha)\| - L \right| \quad (7)$$

A uniform sweep over $\alpha \in [-45, +45]$ with 0.1° resolution selects the angle yielding the minimum residual. A pose is accepted if $e_i < \epsilon = 0.5$ mm for all six legs; otherwise the target is declared unreachable and the motion is aborted. The procedure is summarized in Algorithm 1.

Algorithm 1 Numerical Inverse Kinematics

Require: Pose $\mathbf{q}$, geometry $\{L, r, \mathbf{b}_i, \mathbf{p}_i\}$, tolerance ϵ

Ensure: Servo angles $\{\alpha_i\}$ or FAIL

```

1: Compute  $\mathbf{R}$  from  $(\phi, \theta, \psi)$  via Eq. (2)
2: for  $i = 1$  to 6 do
3:    $\mathbf{P}_i \leftarrow \mathbf{R} \mathbf{p}_i + \mathbf{T}$  ▷ Eq. (4)
4:    $e^* \leftarrow \infty$ 
5:   for  $\alpha = -45$  to  $+45$  step  $0.1$  do
6:      $\mathbf{H} \leftarrow \text{ComputeHornTip}(\mathbf{b}_i, \alpha)$  ▷ Eq. (6)
7:      $e \leftarrow \left| \|\mathbf{P}_i - \mathbf{H}\| - L \right|$ 
8:     if  $e < e^*$  then
9:        $e^* \leftarrow e$ ;  $\alpha_i \leftarrow \alpha$ 
10:    end if
11:  end for
12:  if  $e^* > \epsilon$  then return FAIL
13:  end if
14: end for
15: return  $\{\alpha_i\}$ 
```

D. Trajectory Interpolation

To generate smooth motions, `setPose` linearly interpolates between the current pose $\mathbf{q}_0$ and the target $\mathbf{q}_f$ over N steps at the servo update rate $\Delta t = 20$ ms:

$$N = \max\left(1, \left\lceil \frac{t_{\text{duration}}}{\Delta t} \right\rceil\right), \quad \mathbf{q}_k = (1 - t_k) \mathbf{q}_0 + t_k \mathbf{q}_f, \quad t_k = \frac{k}{N} \quad (8)$$

IK is solved at each interpolated pose. If any intermediate pose is unreachable, motion halts at that step.

IV. Software Architecture

A. Layer Overview

The firmware is structured into well-separated layers, summarized in Table 3.

Table 3 Firmware Layer Breakdown

Layer	Files
Application / Motion sequences	App.cpp, main.cpp
Platform control	StewartPlatform.cpp
Geometry	StewartGeometry.cpp, StewartConfig.h
Servo abstraction	Servo.cpp
PWM driver	PCA9685.cpp
I ² C HAL	i2c.cpp
Linear algebra utilities	linalg.h

B. I²C and PCA9685 Driver

The lowest layer wraps the Arduino `Wire` library into an `i2c` namespace exposing `writeByte` and `readByte` primitives. The `PCA9685` class uses these to configure the `MODE1` register, set the PWM frequency via the prescaler, and write per-channel on/off tick counts. Pulse widths are converted to 12-bit counts by:

$$\text{counts} = \left\lfloor \frac{t_{\mu s} \times 4096}{20000} \right\rfloor \quad (9)$$

assuming the standard 50 Hz (20 ms period) PWM signal.

C. Servo Abstraction

The `Servo` class maps a commanded angle α in degrees to a pulse width using a linear calibration between `minPulse` (−90) and `maxPulse` (+90):

$$t = \frac{\alpha + 90}{180}, \quad \mu s = t_{\min} + (t_{\max} - t_{\min}) t \quad (10)$$

D. Motion Sequences

Two continuous motion sequences are implemented in the `App` class.

Flat Precession. The platform centroid traces a horizontal circle of radius $A = 40$ mm at $f = 0.75$ Hz with no angular component:

$$x(t) = A \cos(2\pi f t), \quad y(t) = A \sin(2\pi f t), \quad z = 175 \text{ mm} \quad (11)$$

Pose updates are dispatched at 50 Hz using a non-blocking elapsed-time check.

Angled Precession. Combines translational and angular circular motion to simulate a precessing gyroscope, with the platform leaning outward in the direction of travel:

$$x(t) = A_p \cos(\omega t), \quad y(t) = A_p \sin(\omega t), \quad \phi(t) = A_\theta \sin(\omega t), \quad \theta(t) = -A_\theta \cos(\omega t) \quad (12)$$

with $A_p = 30$ mm, $A_\theta = 0.2$ rad, $f = 0.25$ Hz, updated at 75 Hz.

V. Results and Future Work**A. System Performance**

The platform successfully demonstrates all six degrees of freedom and executes smooth continuous trajectories. At a 50 Hz servo update rate, motion is visually fluid with no perceptible stepping. The numerical IK solver resolves all six legs comfortably within the 20 ms update budget on the ESP32 running at 240 MHz. Both precession sequences run stably for their full commanded durations without IK failures, confirming that the chosen amplitudes and frequencies keep all interpolated poses within the reachable workspace.

B. Limitations

The numerical IK search has a fixed 0.1° angular resolution. While sufficient for the current application, finer resolution or a closed-form analytical solution would improve accuracy and reduce computational load. The current linear interpolation does not apply velocity smoothing (e.g., trapezoidal or S-curve profiles), so abrupt velocity discontinuities occur at trajectory endpoints. Additionally, the platform joint positions in `StewartConfig.h` currently use `SERVO_RAD` rather than the separate `PLATFORM_JOINT_RAD` constant, a known configuration discrepancy to be corrected.

C. Future Work

Several extensions are planned. First, deriving a closed-form IK solution for this specific geometry would eliminate the search loop and reduce per-update latency. Second, implementing trapezoidal or S-curve velocity profiles in `setPose` would eliminate velocity discontinuities at trajectory boundaries. Third, adding a serial or BLE command interface would enable real-time teleoperation. Fourth, mounting an IMU on the platform and closing the control loop against a reference orientation would enable active stabilization. Finally, sweeping the IK solver across a grid of poses would generate a reachability map useful for motion planning and workspace visualization.

D. Conclusion

This paper has presented a fully functional embedded Stewart Platform using commodity servo hardware and a compact, well-structured C++ firmware stack. The modular layered architecture—from I²C primitives up to named motion sequences—makes the codebase straightforward to extend and adapt to new geometries or motion profiles. The numerical inverse kinematics solver is robust and fast enough for smooth real-time operation on the ESP32, and the two precession sequences demonstrate expressive, continuous 6-DOF motion from a minimal hardware platform.

References

- [1] Stewart, D., “A Platform with Six Degrees of Freedom,” *Proceedings of the Institution of Mechanical Engineers*, Vol. 180, No. 1, 1965, pp. 371–386.
- [2] NXP Semiconductors, “PCA9685: 16-channel, 12-bit PWM Fm+ I²C-bus LED Controller,” Product Data Sheet, Rev. 4, 2015.
- [3] Merlet, J.-P., *Parallel Robots*, 2nd ed., Springer, Dordrecht, 2006.